

**Instructions:** Please answer the following questions, making sure to write legibly on the following pages. The code below is a subsection of a Python file that provides context for the questions below.

- Since we are writing Python code, please be careful about spacing/indentation. Leave enough room that it is obvious when you want things indented.
- Make sure to write legibly.
- All questions are worth equal points unless otherwise stated.
- The quiz is cumulative and may include material from previous weeks.

---

```
from flask import Flask, request, jsonify
from logger_utils import logger # Assume this logger is already imported

app = Flask(__name__)

def calculate_discount(price, discount_percent):
    # Calculate the final price after applying a discount.

    return price * (1 - discount_percent / 100)

def apply_multiple_discounts(price, *discounts):
    # Apply multiple discounts sequentially to a price.

    result = price
    for discount in discounts:
        result = calculate_discount(result, discount)
    return result

@app.route('/api/products/<product_id>/price', methods=['GET'])
def get_product_price(product_id):
    discount = request.args.get('discount')
    base_price = get_base_price(product_id) # Assume this function exists
    final_price = calculate_discount(base_price, discount)
    return jsonify({
        'product_id': product_id,
        'base_price': base_price,
        'discount_percent': discount,
        'final_price': round(final_price, 2)
    }), 200
```

---

1. (3 Points) What does CRUD stand for in the context of API development? Please just write the words, no explanation needed.
  
2. Write a single test function called `test_calculate_discount` that tests the `calculate_discount` function shown on the first page. Assume a logger named `logger` is already imported (as shown in the code context). It should take zero arguments and have no decorators. Your test should:
  - Test that a price of 100 with a 10% discount returns 90.0
  - Test that a price of 50 with a 20% discount returns 40.0
  - Log each successful test result at the INFO level using the format:  
"Test passed: {price}, {discount}, {result}"
  
3. Write a single integration test function called `test_discount_edge_cases` that tests `apply_multiple_discounts` (and so explicitly tests how the functions work together.). Your test should:
  - Test that applying zero discounts (empty list) returns the original price
  - Test that applying a 0% discount produces the same result as applying no discount
  - Test that applying a 100% discount results in a final price of 0.0
  - Log each successful test result using the same format as in Question 2.
  
4. (2 Points) In class we discussed autodocs systems like MkDocs. Please explain why the `site/` directory should *not* be included in a git repository. Write a single sentence answer.